

The Interaction Cycle

Highlights

- Introduction
 - Interaction cycle
 - Need for a theory-based conceptual framework
- Interaction cycle
 - Norman's stages-of-action model of interaction
 - Gulfs between user and system
 - From Norman's model to our interaction cycle
 - Cooperative user-system task performance within the interaction cycle

28.1 INTRODUCTION

28.1.1 What Is the Interaction Cycle?

The interaction cycle is our adaptation of Norman's (1986) "stages-of-action" model that characterizes sequences of user actions typically occurring in interaction between a human user and almost any kind of system.

28.1.2 Need for a Theory-Based Conceptual Framework

As Gray and Salzman (1998, p. 241) have noted:

"To the naïve observer it might seem obvious that the field of HCI would have a set of common categories with which to discuss one of its most basic concepts: Usability. We do not. Instead we have a hodgepodge collection of *do-it-yourself* categories and various collections of *rules-of-thumb*."

As Gray and Salzman (1998) continued, "Developing a common categorization scheme, preferably one grounded in theory, would allow us to compare types of usability problems across different types of software and interfaces." We believe our interaction cycle, along with the concept of affordances, helps meet this need. It provides UX practitioners with a way to frame design issues and UX problem data within the structure of how designs support user actions and intentions.

28.2 NORMAN'S STAGES-OF-ACTION MODEL OF INTERACTION

Norman's (2013) stages-of-action model is illustrated in [Fig. 28.1](#).

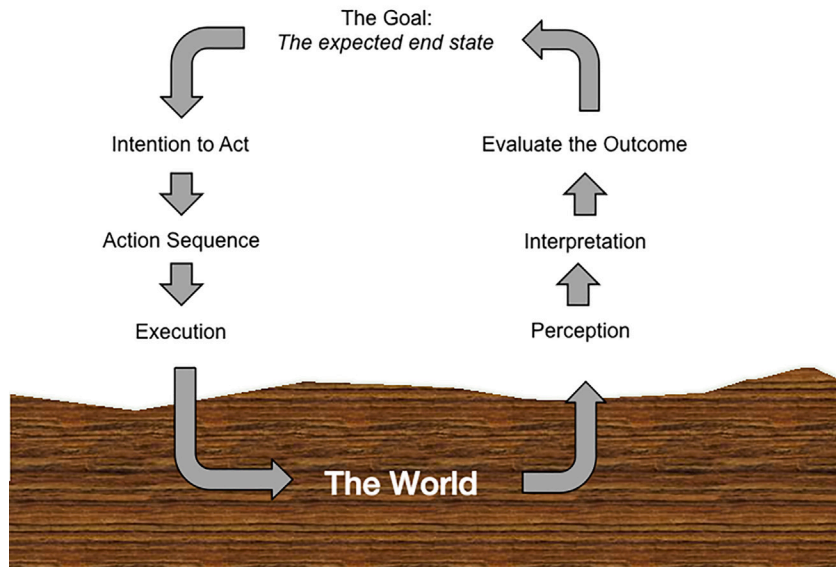


Fig. 28.1

Norman's stages-of-action model (Fig. 2.7 in Norman, [2013]. Courtesy of Don Norman).

The stages of action naturally divide into three major kinds of user activity. On the execution side (left side of [Fig. 28.1](#)), the user typically begins at the top of the figure by establishing a goal, decomposing goals into tasks and intentions, and mapping intentions to action sequence specifications. The user manipulates system controls by executing the physical actions (see [Fig. 28.1](#), bottom left), which cause internal system state changes (outcomes) in the world (the system) at the bottom of the figure.

On the evaluation side (right side of [Fig. 28.1](#)), the user perceives, interprets, and evaluates the outcomes with respect to goals and intentions starting with perceiving the system state by sensing feedback from the system (state changes in the world or the system). Interaction success is then evaluated by comparing outcomes with the original goals. The interaction is successful if the actions have brought the user closer to the goals.

Norman's model, along with the structure of the analytic evaluation method called the cognitive walkthrough (Lewis, Polson, Wharton, et al., 1990), had an essential influence on our interaction cycle. Both ask questions about whether the user can determine what to do with the system to achieve a

goal in the work domain, how to do it in terms of user actions, how easily the user can perform the required physical actions, and (to a lesser extent in the cognitive walkthrough method) how well the user can tell whether the actions were successful in moving toward task completion.

28.2.1 Gulfs Between User and System

Originally conceived by Hutchins, Hollan, and Norman (1986), the gulfs of execution and evaluation were described further by Norman (1986). The two gulfs represent places where interaction can be most difficult for users and where designers need to pay special attention to help users.

28.2.1.1 *The gulf of execution*

Users need help in knowing what actions to make on what objects in the system to accomplish their work goals. The gulf of execution, on the left-hand side of the stages-of-action model in [Fig. 28.1](#), can occur as a language gap from the user's goals in the work domain to actions in the system domain. Users think of their goals in the language of the work domain. To act upon the system to pursue these goals, intentions in the work domain language must be translated into the language of physical actions and the physical system.

As a simple example, consider a user composing a letter with a word processor. The letter is a work domain element, and the word processor is part of the system. The work domain goal of “creating a permanent record of the letter” translates to the system domain intention of “saving the file,” which translates to the action of “clicking on the *Save* icon.” This translation is a mapping between the two domains and is needed to bridge the gulf of execution.

Let us revisit the example of a thermostat on a furnace from [Section 14.2](#). Suppose that a user is feeling chilly while sitting at home. The user formulates a simple goal, expressed in the language of the work domain (in this case, the daily living domain), “to feel warmer.” To meet this goal, something must happen in the physical system domain. The ignition function and air blower of the furnace, for example, must be activated. To achieve this outcome in the physical domain, the user must translate the work domain goal into an action sequence in the physical (system) domain—namely, to set the thermostat up to the desired temperature.

The gulf of execution lies between the user knowing the effect that is desired and what to do to the system to make it happen. In this example, there is a cognitive disconnect for the user in that the physical system variables to be controlled (burning fuel and blowing air) are not the ones the user

cares about (being warm). The gulf of execution can be bridged from either direction—from the user and/or from the system. Bridging from the user’s side means teaching the user about what has to happen in the system to achieve goals in the work domain. Bridging from the system’s side means building help (cognitive affordances) to support the user in the design by hiding the need for translation, keeping the problem couched in work domain language. The thermostat does a little of each—its operation depends on a shared knowledge of how thermostats work, but it also shows a way to set the temperature visually.

To avoid having to train all users, the interaction designer can take responsibility to bridge the gulf of execution from the system’s side by an effective conceptual design to help the user form a correct mental model. Failure of the interaction design to bridge the gulf of execution will be evidenced in observations of hesitation or task blockage before a user action because the user does not know what action to take or cannot predict the consequences of taking an action.

28.2.1.2 *The gulf of evaluation*

The gulf of evaluation, on the right side of the stages-of-action model in [Fig. 28.1](#), can occur as a language gap from the state of things within the system domain to what that implies in the user’s work domain. Specifically, users need help in knowing whether their actions had the expected outcomes. The ability of users to assess outcomes of their actions depends on how well the interaction design supports their comprehension of a change in the system state through their understanding of system feedback.

System state is a function of internal system variables, and it is the job of the interaction designer who creates the display of system feedback to bridge the gulf. This is done by translating a description of system state into the language of the user’s work domain so that outcomes can be compared with the goals and intentions to assess the success of the interaction.

Failure of the interaction design to bridge the gulf of evaluation will be evidenced in observations of hesitation or task blockage after a user action because the user does not understand feedback and does not fully know what happened as a result of the action.

28.2.2 From Norman’s Model to Our Interaction Cycle

We adapted and extended Norman’s (1990) theory of action model of [Fig. 28.1](#) into our interaction cycle, which is also a model of user actions that occur in typical sequences of interaction between a human user and a system.

28.2.2.1 Partitioning the model

Because the early part of Norman's execution side was about planning of goals and intentions, we call it the planning part of the cycle (as adapted in Fig. 28.2). Planning includes formulating goal and task hierarchies, as well as decomposition and identification of specific intentions.

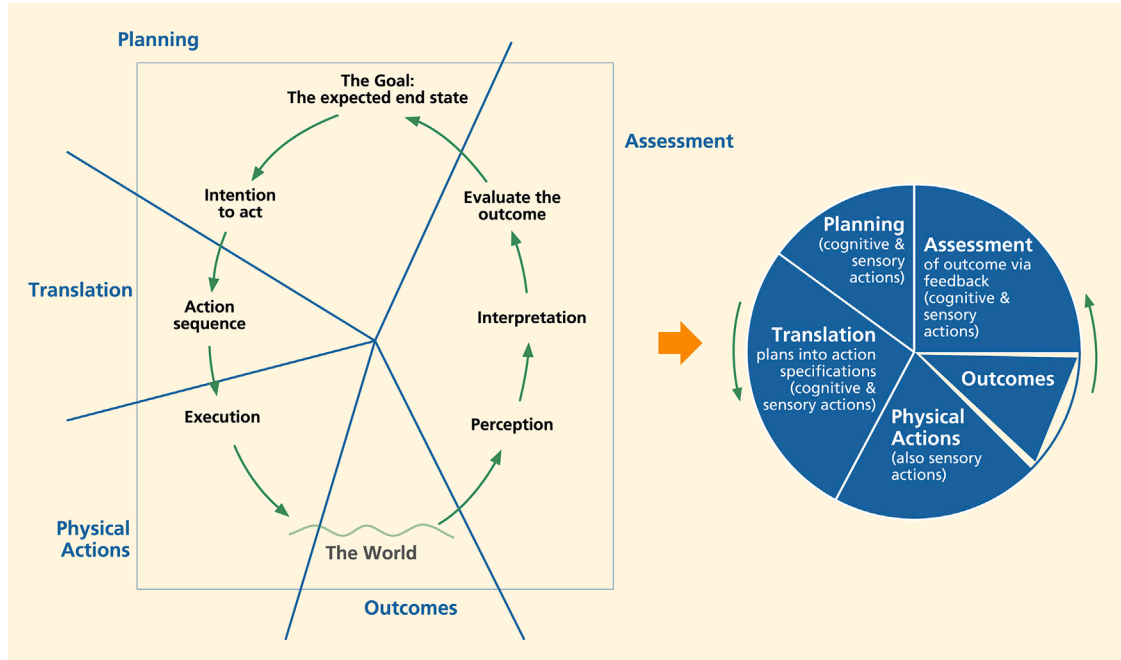


Fig. 28.2

Transition from Norman's model to our interaction cycle.

Planning is followed by formulation of the specific actions (on the system) to carry out each intention, a cognitive action we call translation because it is here that the goals and plan are translated into action intentions.

Norman's "execution" component maps directly into what we call the physical actions part of the interaction cycle. Because Norman's evaluation side is where users assess the outcome of each physical action based on system feedback, we call it the assessment part.

28.2.2.2 Adding outcomes and system response and emphasizing translation

Finally, we added the concepts of outcomes and a system response, resulting in the mapping to the interaction cycle shown in Fig. 28.2. Outcomes are

represented as a floating sector between physical actions and assessment in the cycle because ours is about user interaction, and what happens in outcomes is entirely internal to the system and not part of what the user sees or does. The system response, which includes all system feedback and occurs at the beginning of and as an input to the assessment part, tells users about the outcomes.

Note that the importance of translation to the interaction cycle and its significance in design for a high-quality user experience is, in fact, so great that we made the relative sizes of the wedges of cycle parts represent the weight of this importance visually.

28.3 INTERACTION CYCLE CATEGORIES OF UX DESIGN ISSUES

Here, we give an overview of the kinds of UX design issues that come under each interaction cycle category. These categories and sub-categories, along with the different types of affordances, are used to organize the UX design guidelines in the next chapter.

28.3.1 Planning (Design Helping User Know What to Do)

Planning is the part of the interaction cycle that contains mostly cognitive actions by users to determine certain questions (e.g., “What do I do?” or “What can I do?”) using the system to achieve work domain goals.

Interaction design support for user planning is concerned with how well the design helps users understand the overall computer application relative to the perspective of work context, the work domain, and ecological requirements and constraints to determine in general how to use the system to solve problems and complete work. Planning support has to do with the system model, conceptual design, and metaphors; the user’s awareness of system features and capabilities (what the user can do with the system); and the user’s knowledge of possible system modalities. Planning is about user strategies for approaching the system to complete work.

28.3.2 Translation (Design Helping User Know How to Do Something)

Translation concerns the lowest level of task preparation. Translation includes anything that has to do with deciding how a user can or should make an action on an object, including thinking about which action to take

or on what object to take it, or what might be the best action to take next within a task.

Translation is the part of the interaction cycle that contains all cognitive actions by users to determine how to carry out the intentions that arise in planning, in terms of needed physical actions on interface objects, such as “click and drag the document file icon,” using the computer to carry out an intention.

Here is a simple example about driving a car. Planning for a trip in the car involves thinking about traveling to get somewhere, involving specific questions (e.g., “Where are you going?” and “By what route?” and “Do we need to get gas or groceries first?”). So planning is in the work domain, which is travel, not operating the car.

In contrast, translation takes the user into the system, machine, or physical world domain and is about formulating actions to operate the gas pedal, brake, and steering wheel to accomplish the tasks that will help you reach your planning, or travel, goals. Because steps such as “turn on headlight switch,” “push horn button,” and “push brake pedal” are actions on objects, they are the stuff of translation.

Over the bulk of interaction that occurs in the real world, translation is arguably the single-most important part of the cycle because it is about how you do things. That is why designing cognitive affordances for effective feed-forward in translation is so important, as we’ll see in [Section 29.6](#). From our own experiences over the years, the largest bulk of UX problems observed in UX evaluation ($\geq 75\%$) falls into this category.

28.3.3 Physical Actions (Design Helping User Do the Actions)

After users decide which actions to take, the physical actions part of the interaction cycle is where users do the actions. This part includes all user inputs acting on devices and user interface (UI) objects to manipulate the system, including typing, clicking, dragging, touching, gestures, and navigational actions, such as walking in virtual environments. The physical actions part of the interaction cycle includes no cognitive actions; thus this part is not about thinking about the actions or determining which actions to do.

Physical actions are especially important for analysis of performance by expert users who have, to some extent, automated planning and translation associated with a task and for whom physical actions have become the limiting factor in task performance.

Physical affordance design factors include design of input/output devices (e.g., touchscreen design or keyboard layout), haptic devices, interaction styles and techniques, direct manipulation issues, gestural body movements, physical fatigue, and such physical human factors issues as manual dexterity, hand–eye coordination, layout, interaction using two hands and feet, and physical disabilities.

28.3.4 Outcomes (Internal, Invisible Effect/Result Within System)

Physical user actions are seen by the system as inputs and usually trigger a system function that can lead to system state changes that we call outcomes of the interaction. The outcomes part of the interaction cycle represents the system’s turn to do something that usually involves computation by the non-UI software or, as it is sometimes called, core or backend functionality. One possible outcome is a failure to achieve an expected or desired state change, as in the case of a system error.

A user action is not always required to produce a system response. The system can also autonomously produce an outcome, possibly in response to an internal event such as a disk becoming full, an event in the environment sensed by the system such as a process control alarm, or the physical actions of other users in a shared work environment.

The system functions that produce outcomes are purely internal to the system and do not involve the user. Consequently, outcomes are technically not part of the user’s interaction cycle, and the only UX issues associated with outcomes may be about usefulness or functional affordance of the non-UI system functionality.

Because internal system state changes are not directly visible to the user, outcomes must be revealed to the user via system feedback or a display of results, to be evaluated by the user in the assessment part of the interaction cycle.

28.3.5 Assessment (Design Helping User Know if Interaction Was Successful)

The assessment part of the interaction cycle corresponds to Norman’s (1990) evaluation side (see [Fig. 28.1](#)) of interaction. A user in assessment performs sensory and cognitive actions needed to sense and understand system feedback and displays results as a means to comprehend internal system changes or outcomes due to a previous physical action or other triggers to system change.

The user's objective in assessment is to determine whether the outcomes of all previous planning, translation, and physical actions were favorable, meaning desirable or effective to the user. In particular, an outcome is favorable if it helps the user approach or achieve the current intention, task, and/or goal—that is, if the plan and action worked.

The assessment part parallels much of the translation part, only focusing on system feedback. Assessment has to do with the existence of feedback, presentation of feedback, and content or meaning of feedback. Assessment is about whether users can know when an error occurred, and whether a user can sense a feedback message and understand its content. The necessity of designing cognitive affordances for effective feedback in assessment is reflected in the importance of guidelines for the same in [Section 29.9](#).

Example: Creating a Business Report as a Task Within the Interaction Cycle

Let us say that the task of creating a quarterly report to management on the financial status of a company breaks down into these basic steps:

1. Calculate monthly profits for last quarter.
2. Write summary, including graphs, to show company performance.
3. Create table of contents.
4. Print the report.

In this kind of task decomposition, it is common that some steps are more or less granular than others, meaning that some steps will decompose into more sub-steps and more details than others. As an example, the first step can decompose into these sub-steps:

1. Open spreadsheet program.
2. Call accounting department and ask for numbers for each month.
3. Create column headers in spreadsheet for expenses and revenues in each product category.
4. Compute profits.

The first step here, to open a spreadsheet program, may correspond to a single simple pass through the interaction cycle. The second step is a non-system task that interrupts the workflow temporarily. The third and fourth steps could take several passes through the interaction cycle.

The first intention for the fourth task step, “Print report,” is the “getting started” intention; the user intends to invoke the print function, taking the

task from the work domain into the computer domain. In this particular case, the user does no further planning at this point, expecting the feedback from acting on this intention to lead to the next natural intention.

To translate this first intention into an action specification in the language of the actions and objects within the computer interface, the user draws on experiential knowledge and/or the cognitive affordances provided by display of the **Print** choice in the **File** menu to create the action specification to select **Print**. A more experienced user may translate the intention subconsciously or automatically to the shortcut action **Ctrl-P** or click on the printer icon.

The user then carries out this action specification by doing the corresponding physical action, clicking the **Print** menu choice. The system accepts the menu choice, changes state internally (the outcomes of the action), and displays the **Print** dialog box as feedback. The user sees the feedback and uses it for assessment of the outcome thus far. Because the dialog box makes sense to the user at this point in the interaction, the outcome is considered to be favorable—that is, leading to accomplishment of the user’s intention and indicating successful planning and action so far.

28.4 COOPERATIVE USER-SYSTEM TASK PERFORMANCE WITHIN THE INTERACTION CYCLE

28.4.1 Primary Tasks

Primary tasks have direct work-related goals. The task in the previous example of creating a business report is a primary task, as is the small task of printing. Primary tasks can be user initiated or initiated by the environment, the system, or other users. Primary tasks usually represent simple, linear paths through the interaction cycle.

User-initiated tasks. The typical linear path of user-system turn taking starting at the top and going counterclockwise around the interaction cycle represents a user-initiated task because it starts with user planning and translation.

Tasks initiated by environment, system, or other users. When a user task is initiated by events that occur outside that user’s interaction cycle, the user actions become reactive. The user’s cycle of interaction begins at the outcomes part, followed by the user sensing the subsequent system response in a feedback output display and thereafter reacting to it.

28.4.2 Path Variations in the Interaction Cycle

For all but the simplest tasks, interaction can take alternative, possibly nonlinear, paths. Although user-initiated tasks usually begin with some kind of planning, Norman (1986) emphasized that interaction may not follow a simple cycle of actions. Some activities will appear out of sequence or be

omitted or repeated. In addition, the user's interaction process can flow around the interaction cycle at almost any level of task/action granularity.

Multi-user tasks. When the interaction cycles of two or more users interweave in a cooperative work environment, one user will enter inputs through physical actions (Fig. 28.3), and another user will assess (sense,

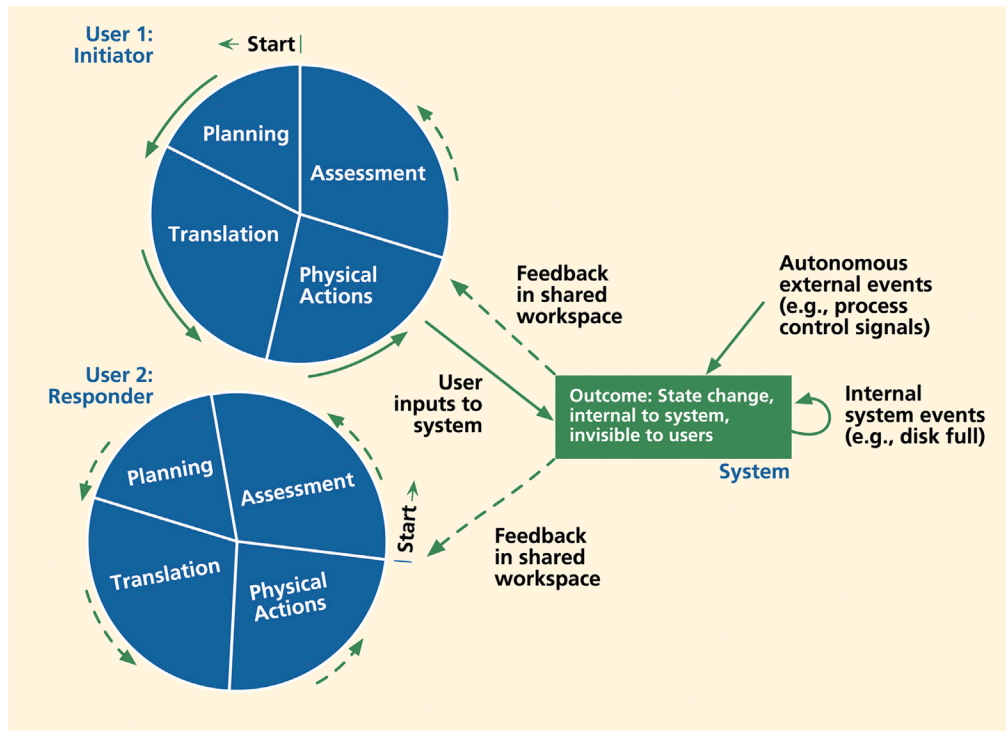


Fig. 28.3

Multi-user interaction, system events, and asynchronous external events within multiple interaction cycles.

interpret, and evaluate) the system response, as viewed in a shared workspace, revealing system outcomes due to the first user's actions. For the user who initiated the exchange, the cycle begins with planning, but for the other users, interaction begins by sensing the system response and goes through assessment of what happened before it becomes the second user's turn for planning for the next round of interaction.

28.4.3 Secondary Tasks, Intention Shifts, and Stacking

Secondary tasks and intention shifts. Kaur, Maiden, and Sutcliffe (1999), who based an inspection method primarily for virtual environment applications

on Norman's (1990) stages of action, recognized the need to interrupt primary work-oriented tasks with secondary tasks to accommodate intention shifts, exploration, information seeking, and error handling. They created different and separate kinds of interaction cycles for these cases, but from our perspective, these cases are just variations in flow through the same cycle.

Secondary tasks are overhead tasks in that the goals are less directly related to the work domain and usually oriented more toward dealing with the computer as an artifact, such as error recovery or learning about the interface. Secondary tasks often stem from changes of plans or intention shifts that arise during task performance; something happens that reminds the user of other things that need to be done or that arise out of the need for error recovery.

For example, the need to explore can arise in response to a specific information need, when users cannot translate an intention to an action specification because they cannot see an object that is appropriate for such an action. Then they have to search for such an object or cognitive affordance, such as a label on a button or a menu choice, that matches the desired action.

Stacking and restoring task context. Stacking and restoring of work context during the execution of a program is an established software concept. Humans must do the same during the execution of their tasks due to spontaneous intention shifts. Interaction cycles required to support primary and secondary tasks are just variations of the basic interaction cycle; however, the storing and restoring of task contexts in the transition between such tasks impose a human memory load on the user, which could require explicit support in the interaction design. Secondary tasks also often require considerable judgment on the part of the user when it comes to assessment. For example, an exploration task may be considered concluded when users are satisfied that they have had enough or get tired and give it up.

Example of stacking due to intention shift. To use the previous example of creating a business report to illustrate stacking due to a spontaneous intention shift, suppose the user has finished the task step "Print the report" and is ready to move on. However, upon looking at the printed report, the user does not like the way it turned out and decides to reformat the report. The user has to take some time to learn more about how to format it better. The printing task is stacked temporarily while the user takes up the information-seeking task.

Such a change of plan causes an interruption in the task flow and normal task planning, requiring the user mentally to stack the current goal, task, and/or intention while tending to the interrupting task (Fig. 28.4). As the primary task is suspended in mid-cycle, the user starts a new interaction cycle at planning for the secondary task. Eventually, the user can unstack each goal and task successively and return to the main task.

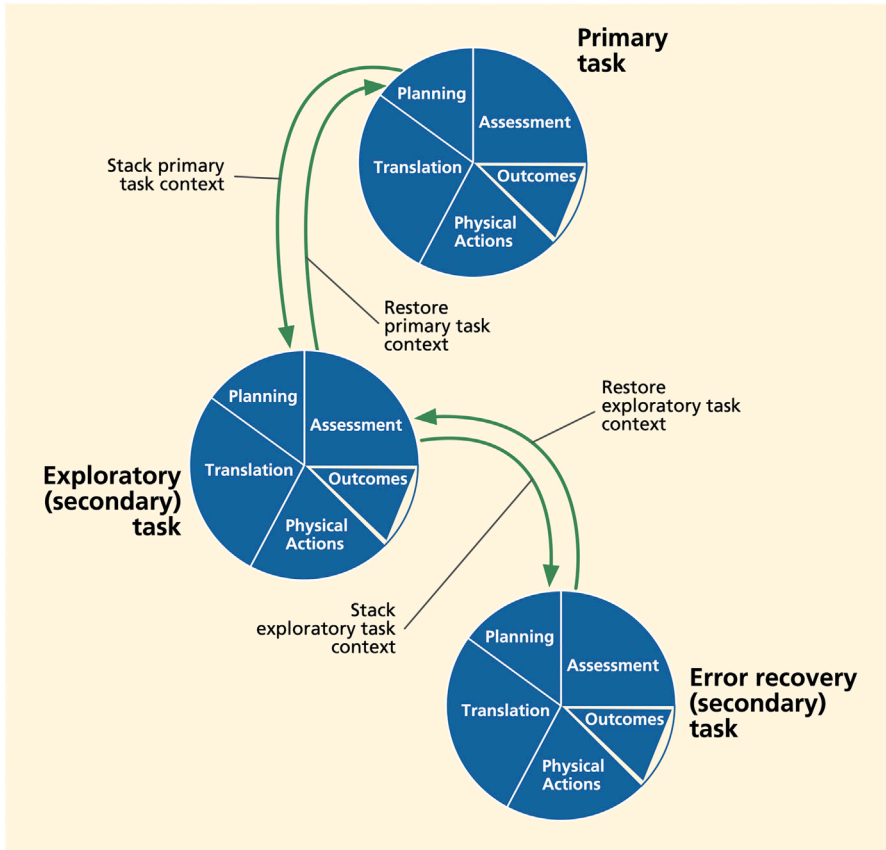


Fig. 28.4

Stacking and returning to interaction cycle task context instances.

28.5 SEQUE

In this chapter, we discussed the interaction cycle and its basic stages. In the next chapter, we use the interaction cycle as a way to organize UX design guidelines.